

Skynet MMO 学习课程第二课： Service 模型与 Actor 架构

基线：Skynet v1.8.0；Bundled Modified Lua 5.4.7；本仓库 `main` 分支。

前置能力：已经完成第一课，能够解释 `skynet -> snlua -> loader.lua -> bootstrap.lua -> service/main.lua`，并能使用 LuaPanda、Debug Console 和 GDB。

本课目标：建立足以做架构评审和 coroutine 正确性审查的 Service 心智模型，而不止会调用 `skynet.newservice/call/send`。

SKYNET MMO LEARNING

目录

0. 先给结论：本课要纠正的核心误解	4
1. 从第一课接过来：Service 是怎样存在的	5
1.1 五个概念不能混为一谈	5
2. Actor 模型在 Skynet 中具体落在哪里	6
2.1 Actor 边界的价值	6
3. Mailbox 与 Worker：串行的到底是什么	7
3.1 “同一 Service 不被两个 Worker 同时执行” 不等于 “业务事务不重入”	7
4. 一条消息实际携带什么	8
5. skynet.dispatch：每条请求为何是一条 coroutine	9
5.1 为什么 LuaPanda 加载顺序会影响业务断点	9
6. send 与 call：不是异步和同步函数调用那么简单	10
6.1 skynet.send	10
6.2 skynet.call	10
6.3 选择规则	10
7. Service coroutine 重入：用登录代码看真实竞态	11
8. 为什么 PlayerAgent 还需要 skynet.queue	12
8.1 它保证什么，不保证什么	12
9. 状态所有权：PlayerAgent 与 Scene 为什么不是双主	13
9.1 Scene 中为什么 move 故意不 yield	13
10. Service 边界：Manager、Worker 与 Hot Actor	14
10.1 Manager 应管理控制面，不应永久代理数据面	14
10.2 不要把每个 module 做成 Service	14
10.3 Hot Actor 的信号	14
11. call 链、失败传播与死锁	15
11.1 call 失败不等于业务返回 false	15
12. 生命周期与陈旧消息：fd、Service Address、timer 都会过期	16
12.1 Connection ID 防 fd 复用	16
12.2 Offline Version 防旧 timer	16
13. 调试实验一：亲眼观察 call 的 yield/resume	17

13.1 准备断点	17
13.2 切换到 Scene 验证 owner	17
14. 调试实验二：用 Debug Console 看 Actor，而不是只看日志	18
15. 调试实验三：C Runtime 断点	19
16. 设计练习：三个看似合理但有问题的方案	20
16.1 方案 A：所有战斗请求经过 CombatMgr	20
16.2 方案 B：PlayerAgent 先改坐标，再通知 Scene	20
16.3 方案 C：Scene 移动中 call NavService 查询碰撞	20
17. 修改 Service 前的强制审查模板	21
17.1 状态所有权	21
17.2 执行上下文与 yield	21
17.3 消息选择	21
17.4 架构与生命周期	21
17.5 验证	21
18. 常见误判速查	22
19. 本课源码索引	23
20. 自测题	24
1. 为什么“一个 Service 一条 Mailbox”仍不足以证明业务 handler 原子执行？	24
2. skynet.call 如何把 Response 恢复到正确 coroutine？	24
3. 为什么 Scene 的 CMD.move 中 skynet.send Push 不破坏当前提交段的原子性？	24
4. PlayerAgent 已有 Service Mailbox，为什么还对客户端请求使用 skynet.queue？	24
20.1 自测题（续）	25
5. 为什么坐标同时出现在 PlayerAgent 和 Scene 中却仍可维持单一权威？	25
6. 如何判断一个新模块是否应拆成 Service？	25
7. 找到一个 skynet.call 后，代码评审至少还要追什么？	25
8. 增加 Worker Thread 为什么通常不能修复单个 Scene Mailbox 堆积？	25
21. 本课完成标准与第三课入口	26

0. 先给结论：本课要纠正的核心误解

最危险的说法是：

“Skynet 的 Service 就是单线程 Actor，所以 Service 内不需要考虑并发。”

这句话只有一半正确。

- 一个 Service 有自己的地址、状态和 Mailbox；Runtime 不会让两个 Worker Thread 同时进入同一个 `skynet_context` 的 C callback。
- 但一条 Lua 消息通常运行在一条 coroutine 中。它在 `skynet.call`、`skynet.sleep`、`skynet.wait` 等位置 yield 后，Service 可以处理下一条消息。
- 因而，同一 Service 内的两条业务 coroutine 可以在时间上交错。yield 前读出的状态，恢复后可能已经失效。

对 C++ MMO Server 更准确的类比不是“固定线程上的永不重入 Game Loop”，而是：

```
Service = Actor Address + Private Mailbox + State + Message Dispatcher
Lua message handler = ■■■■■■ Fiber/Coroutine
Worker Thread = ■■■■■■■■ Service ■■■■■■
```

本课所有讨论最终落到一句审查问题：

当前 coroutine yield 以后，同一 Service 的哪条消息能修改我刚才读过的状态？

1. 从第一课接过来：Service 是怎样存在的

第一课已经走到 `skynet.newservice("scene/scene")`。现在把这句代码展开。

在 Lua 层，`third_party/skynet/lualib/skynet.lua` 的 `skynet.newservice` 并不直接 new 一个 Lua 对象：

```
function skynet.newservice(name, ...)
    return skynet.call(".launcher", "lua", "LAUNCH", "snlua", name, ...)
end
```

这里至少发生四件事：

1. 当前 Service 向 `.launcher` 发出同步请求，并挂起当前 coroutine。
2. `.launcher` 请求 Runtime 创建一个 `snlua` C Service。
3. `snlua` 创建独立 Lua State，再由 `loader.lua` 加载目标 Lua 文件。
4. 新 Service 初始化成功后，`.launcher` 回复 Service Address，调用方 coroutine 才恢复。

所以一个 Lua Service 不是 class instance，也不是一个 coroutine。它更接近一个隔离的 Actor Runtime 单元：拥有 `skynet_context`、Service Address、私有消息队列、Lua State，以及在该 Lua State 中创建的一组 coroutine。

1.1 五个概念不能混为一谈

概念	数量关系	主要职责	本仓库例子
Linux Process	通常一个	承载整个 Skynet Node	<code>third_party/skynet/skynet</code>
Worker Thread	<code>thread</code> 配置决定	从 Global Queue 取可运行 Service Queue	<code>config/game.lua</code> 中 <code>thread = 8</code>
Service / <code>skynet_context</code>	多个	地址、Mailbox、callback、生命周期、统计	PlayerAgent、Scene、Watchdog
Lua State	每个 <code>snlua</code> Service 一个	Lua 全局环境、module cache、GC Heap	<code>service/scene/scene.lua</code> 所在状态
coroutine	每个 Lua Service 多条	执行消息 handler，允许 yield/resume	<code>REQUEST.move</code> 所在 coroutine

两个直接推论：

- 不同 Service 的 Lua table 不共享；传递的是序列化消息，不是跨 Lua State 指针。
- Worker Thread 会变化。不要把 TLS、线程编号或“本帧线程”当成某个 Service 的稳定身份。

2. Actor 模型在 Skynet 中具体落在哪里

Actor 不是“把所有东西做成 Service”。在本课语境中，一个合格 Service 至少同时具有四个属性：

1. 身份：可通过 Service Address 定位。
2. 隔离状态：状态只由该 Service 的代码直接读写。
3. 消息边界：外部只能通过消息请求行为，不能持有内部对象引用。
4. 独立生命周期：可以创建、初始化、退出并被监控。

Skynet 的 Service Address 通常以整数 Handle 存在，日志和 Debug Console 常显示成 `:0100000f` 这样的十六进制地址。`.launcher`、`.service` 这类以点开头的是本 Node 内注册名。

Service Address 是路由身份，不是 C++ 指针。目标退出后，旧地址不可继续被当作有效对象引用；涉及 fd、Service Address、timer callback 的延迟消息，都要考虑 Generation/Version Check。

2.1 Actor 边界的价值

一个边界值得成为 Service，通常因为它提供至少一项实质价值：

- 明确唯一状态所有者，例如 Scene 拥有空间与战斗状态。
- 隔离故障或生命周期，例如每个在线玩家一个 PlayerAgent。
- 提供真实并行度，例如不同 Scene 可由不同 Worker 执行。
- 隔离阻塞或外部资源，例如 Storage Worker 管理 DB connection。
- 形成自然的容量或部署分片边界。

“代码很多”“想分层”“这个 module 名字听起来像 Manager”都不是拆 Service 的充分理由。纯算法模块 `scene/aoi_grid.lua` 和 `scene/movement.lua` 留在 Scene Lua State 内，以普通 module composition 复用，比再套一层 RPC 更直接。

3. Mailbox 与 Worker：串行的到底是什么

Runtime C 层的关键结构在：

- `third_party/skynet/skynet-src/skynet_server.c`: `struct skynet_context`、消息投递和 dispatch。
- `third_party/skynet/skynet-src/skynet_mq.c`: 每个 Service 的 `message_queue` 与全局可运行队列。

可以把结构压缩成下面这张图：



`skynet_context_push` 先找到目标 `skynet_context`，再调用 `skynet_mq_push`。

`message_queue` 用 Spinlock 保护 ring buffer；队列需要调度时才进入 Global Queue，避免同一个 Service Queue 被重复并发领取。

Worker 通过 `skynet_context_message_dispatch` 处理队列。它可能按 weight 一次消费多条消息，也会把仍有积压的队列重新放回 Global Queue，兼顾吞吐与 Service 间公平性。

3.1 “同一 Service 不被两个 Worker 同时执行”不等于“业务事务不重入”

C callback 的这次进入确实是串行的。但 Lua dispatcher 收到请求后，会创建或复用一条 coroutine。若 coroutine yield，C callback 已经返回给 Runtime；下一条 Mailbox 消息随后可以进入同一 Lua State，并运行另一条 coroutine。

时间线如下：

```

Worker A: Message A -> coroutine A -> read state.version = 7
          -> skynet.call(B) -> yield

Worker B: Message B -> coroutine B -> state.version = 8 -> return

Worker C: Response from B -> resume coroutine A
          -> coroutine A version 7
  
```

Worker A/B/C 可以是同一条物理线程，也可以不同；竞态成立与否不依赖线程是否变化。问题来自 coroutine 交错与共享 Service State，而不是 Data Race 意义上的同时写内存。

4. 一条消息实际携带什么

Runtime 的核心消息可以抽象成：

```
source   : ■■ Service Address
session  : ■■/■■■■■■■■send ■■■ 0
type     : PTYPE_LUA■PTYPE_RESPONSE■PTYPE_CLIENT ■
payload  : ■■■■■■■■■■
```

在 `skynet_server.c` 中，消息 type 被编码在 `msg->sz` 高位，payload size 位于低位；`dispatch_message` 拆出二者后调用目标 `skynet_context` 注册的 callback。

对于 `snlua`，这个 callback 经过 `lua-lib-src/lua-skynet.c` 的 `_cb`，最终调用 `skynet.dispatch_message(type, msg, sz, session, source)`。之后才进入 `lua-lib/skynet.lua` 的协议表和 Lua coroutine 调度。

这是定位问题时的重要分层：

```
■■■■■■ Mailbox      -> ■■■■■■■■Socket/Gate ■■■ C Runtime
■■■■■■ Lua handler ■■ -> callback■protocol id■unpack/dispatch ■■
handler ■■■■■■      -> coroutine yield■■■ call■■■■■■■
■■■■■■■■ coroutine -> session ■■■■■■■■■■■■
```

5. skynet.dispatch: 每条请求为何是一条 coroutine

业务代码通常注册:

```
skynet.dispatch("lua", function(session, source, command, ...)
    local fn = assert(CMD[command])
    local result = { fn(...) }
    if session ~= 0 then
        skynet.retpack(table.unpack(result))
    end
end)
```

`skynet.dispatch` 只是把函数放进某个 protocol descriptor。真正收到非 Response 消息时，`raw_dispatch_message` 会:

1. 根据 `prototype` 找到协议。
2. `co_create(f)` 得到消息处理 coroutine。
3. 记录 `session_coroutine_id[co] = session`。
4. 记录 `session_coroutine_address[co] = source`。
5. unpack payload, 并 `coroutine_resume` 进入业务 dispatcher。

因此 `session` 和 `source` 不是装饰参数: 它们决定 `skynet.ret/retpack` 应回复给谁, 以及哪一个请求会被解除等待。

5.1 为什么 LuaPanda 加载顺序会影响业务断点

`skynet.lua` 会缓存 `coroutine.create`。LuaPanda 必须先包装 coroutine API, 再加载 `skynet`, 否则顶层代码可能命中, 后续由 Skynet 创建的消息 coroutine 却不受 Debug Hook 跟踪。本仓库通过 `lua-lib/debug/luapanda_preload.lua` 和回归测试固定了这一顺序。

这不是 IDE 路径问题, 而是 Runtime 如何创建业务执行上下文的问题。

6. send 与 call：不是异步和同步函数调用那么简单

6.1 skynet.send

```
skynet.send(addr, "lua", "push", name, args)
```

`send` 使用 `session = 0` 投递消息。发送成功只代表消息已交给 Runtime 路由，并不代表目标已经执行成功。调用方不 `yield`，不等待返回。

适合：

- 通知和 Push。
- 调用方不需要结果，且失败由监控/幂等/补偿另行处理。
- 高频数据面中可以接受异步处理的路径。

6.2 skynet.call

```
local ok, rx, ry = skynet.call(scene, "lua", "move", player_id, x, y)
```

`call` 先分配非零 `session`，发送请求，再在 `yield_call` 中登记：

```
watching_session[session]      = target service
session_id_coroutine[session] = current coroutine
coroutine_yield("SUSPEND")
```

目标 `skynet.retpack` 发回 `PTYPE_RESPONSE + session`。调用方收到 Response 后，从 `session_id_coroutine` 找到原 `coroutine` 并 `resume`。

因此 `call` 的真实语义是：

发送带关联 `session` 的消息，并挂起当前 `coroutine`；不是阻塞整个 Service，也不是阻塞某条固定 Worker Thread。

6.3 选择规则

每次写 `call` 前问：调用方是否必须在当前业务结果中使用对方返回值？若否，优先考虑 `send`。若是，再检查：

- `yield` 前读过哪些本地状态？
- 恢复时目标是否可能退出？
- 是否形成 `A -> B -> C -> D` 的深 RPC 链？
- 上游超时/断连后，下游工作是否还值得继续？
- Mailbox 积压会不会沿同步链放大为 Tail Latency？

7. Service coroutine 重入：用登录代码看真实竞态

`service/gateway/watchdog.lua` 的登录流程在第一次 `skynet.call` 前先执行：

```
c.state = "AUTHING"
local auth_ok = skynet.call(auth_service, "lua", "verify", ...)
if not alive(fd, c) then
    return
end
```

这里包含两个关键动作。

第一，在 `yield` 前发布状态变化。如果先 `call` 再把状态改成 `AUTHING`，第二个 Socket Packet 可能在第一条 coroutine 挂起时进入，并再次启动登录。

第二，恢复后重新验证对象身份。认证期间连接可能已经关闭，`fd` 甚至可能被 OS 复用。`alive(fd, c)` 不能只看 `fd`，还要确认 `table` 中仍是同一个 connection object / generation。

对 C++ Fiber Server，可把它看成典型规则：

```
Validate -> Publish in-flight state -> Await -> Revalidate generation -> Commit
```

错误写法：

```
local c = connections[fd]
local ok = skynet.call(auth, "lua", "verify", ...)
c.state = "PLAYING" -- c ■■■■ close/reuse ■■■■
```

正确性不是靠“Service 单线程”获得的，而是靠 `yield` 边界前后的显式状态协议获得的。

8. 为什么 PlayerAgent 还需要 `skynet.queue`

PlayerAgent 的职责是“同一玩家的客户端业务操作保持串行”。但每个网络请求进入 `PTYPE_CLIENT` dispatcher 时，也是在独立 coroutine 中执行；`REQUEST.move` 内部会 `skynet.call(Scene)` 并 `yield`。

如果没有额外串行器：

```
move #1 -> call Scene -> PlayerAgent coroutine #1 yield
attack -> call Scene -> PlayerAgent coroutine #2 yield
move #2 -> call Scene -> PlayerAgent coroutine #3 yield
```

Scene Mailbox 当前可能仍按到达顺序处理，但 PlayerAgent 自己的请求级不变量、响应顺序、持久快照更新就不再天然是一条完整串行事务。未来任何新增 `await` 都可能改变时序。

本仓库用：

```
local queue = require "skynet.queue"
local serial = queue()

local function dispatch_request(name, args)
    return serial(REQUEST[name], args)
end
```

`skynet.queue` 是 coroutine mutex。持有者即使在函数内部 `yield`，其他 coroutine 也会在 `skynet.wait()` 处排队；释放时 `skynet.wakeup()` 下一个等待者。

8.1 它保证什么，不保证什么

它保证经过同一个 `serial(...)` 入口的临界区互斥，包括跨 `yield` 的互斥。它不保证：

- 绕过 `serial` 的 `CMD.push/client_closed/timeout` 不会运行。
- Scene 内状态自动与 PlayerAgent 一致。
- 下游 RPC 不会死锁或长时间阻塞。
- 多个 PlayerAgent 之间有全局顺序。

所以审查时必须列出所有 dispatcher、timer 和 fork 入口，而不是看到一个 `skynet.queue` 就宣布线程安全。

9. 状态所有权：PlayerAgent 与 Scene 为什么不是双主

本仓库采用：

- PlayerAgent 拥有持久玩家状态：身份、等级、金币、最终需要保存的角色快照，以及当前连接绑定。
- Scene 拥有实时空间/战斗状态：权威坐标、移动额度、AOI 可见集合、怪物 HP、战斗结果。

`PlayerAgent.player.x/y` 是 Scene 成功提交后更新的持久化快照，不是移动判定权威来源。

真实 `move` 路径：

```
Client: move 102 100
-> Gate ■ forward ■ PTYPE_CLIENT ■■
-> PlayerAgent protocol dispatch
-> serial(REQUEST.move)
-> skynet.call(Scene, "move", player_id, 102, 100) [yield]
-> Scene.CMD.move
    movement.try_move
    grid:move
    update authoritative x/y + visible set
    send AOI pushes
    retpack(true, x, y)
-> PlayerAgent coroutine resume
-> update persistence snapshot player.x/y
-> encode client response
```

如果 Scene 拒绝移动，PlayerAgent 不修改快照。这样“校验失败绝不污染权威坐标/AOI/持久化快照”是一条清晰不变量。

9.1 Scene 中为什么 `move` 故意不 `yield`

`service/scene/scene.lua` 从读取 `player`、校验移动额度，到更新 `Grid`、权威坐标和 `visible` 集合之间没有 `call/sleep/wait`。其中向 PlayerAgent 广播使用 `skynet.send`，不会挂起当前 `coroutine`。

于是一次移动提交在 Scene Service 语义下是不可插入的临界段。若未来在中间加入同步 RPC，例如查询外部碰撞 Service，就必须重新设计：

- `yield` 前是否预留移动额度？
- 恢复后 `player` 是否还在 Scene？
- 地图障碍版本是否变化？
- 旧 `visible` 是否仍可用于 `diff`？
- 失败如何回滚 `Grid` 与坐标？

“只加一条 `skynet.call`”可能把原本原子的状态提交拆成两个可重入阶段。

10. Service 边界：Manager、Worker 与 Hot Actor

10.1 Manager 应管理控制面，不应永久代理数据面

`SceneMgr` 负责 Scene 的创建、查找和进场路由；`PlayerAgent` 获得 Scene Address 后，移动直接 call Scene。若所有 `move/attack` 永远先经过 `SceneMgr`：

```
PlayerAgent -> SceneMgr -> Scene
```

`SceneMgr` 会增加一次序列化、一次 Mailbox 排队和一段同步 RPC 链，并成为所有地图的中心热点。Manager 应尽快把稳定目标地址交给调用方，让高频数据面直达 owner。

10.2 不要把每个 module 做成 Service

下面这些更适合普通 Lua module：

- 纯函数战斗公式。
- AOI Grid 数据结构。
- Sproto 编解码辅助。
- 只被单一 owner 使用的状态机子模块。

拆 Service 会引入消息复制、序列化、异步失败、生命周期和监控成本。只有当隔离、并行、资源所有权或独立生命周期的收益超过这些成本时才拆。

10.3 Hot Actor 的信号

- `stat` 中 Mailbox 长期增长或频繁出现 `May overload`。
- 单个 Service CPU 占比高，且大量无关实体都必须经过它。
- 同步调用者大量挂起在同一目标。
- Manager 名义上只路由，实际承载所有高频请求。

常见拆分方向是按天然 owner 分片：Scene Instance、Player、Guild、Match，而不是在 Service 内随意加锁或增加 Worker 数。增加 Worker 不能让单个不可并行 Actor 自动并行。

11. call 链、失败传播与死锁

考虑：

```
Watchdog -> PlayerMgr -> PlayerAgent -> StorageMgr -> StorageWorker
```

每个箭头若都是 `call`，上游 coroutine 都保持等待状态。链越深：

- 任一 Mailbox 排队会叠加到端到端延迟。
- 下游退出会通过 Error Message 唤醒等待者，但业务补偿仍需显式设计。
- 更容易形成环形等待：A call B, B 又 call A。
- Debug Console `task` 会看到大量挂起 coroutine，但根因可能在最下游。

Skynet 不会替业务检测分布式调用环。设计审查时画出同步等待图；若存在有向环，就需要改为 `send + state machine`、调整 owner，或让响应沿原链返回而不是反向 call。

11.1 call 失败不等于业务返回 false

需要区分：

- 目标正常 `retpack(false, reason)`：RPC 成功完成，业务拒绝。
- 目标不存在、退出或未返回：Runtime/Lua 调用失败，`skynet.call` 可能抛错。
- 目标 handler 内异常：目标记录 traceback，请求方收到失败路径。
- 目标活着但长期排队：没有立即异常，只表现为 coroutine 长时间挂起和 Tail Latency。

不要用宽泛 `pcall` 把所有情况吞成同一个 `false`。错误类型决定是否重试、回滚、踢线、报警或直接暴露程序缺陷。

12. 生命周期与陈旧消息：fd、Service Address、timer 都会过期

Actor 系统里“消息发送时有效”不代表“处理时仍有效”。本仓库有两个典型 Generation Check。

12.1 Connection ID 防 fd 复用

PlayerAgent 的 `client_closed(fd, connection_id)` 同时比较 fd 与逻辑连接代号。只比较 fd 不安全，因为 OS Descriptor 可以在旧 close 消息抵达前被新连接复用。

12.2 Offline Version 防旧 timer

掉线时保存 `version = offline_version`，60 秒后 callback 再比较版本和当前 `client_fd`。玩家若期间重连，版本已变化，旧 timer 必须失效。

通用模式：

```
local version = object.version
skynet.timeout(delay, function()
    if object.version ~= version then
        return -- stale timer/message
    end
    commit()
end)
```

同样的审查适用于：Scene Transfer、Match 重建、Guild Leader Lease、异步 DB 回调和缓存刷新。Generation 是逻辑身份的一部分。

13. 调试实验一：亲眼观察 `call` 的 `yield/resume`

13.1 准备断点

在 Windows VS Code 打开 WSL Workspace，选择 Skynet Lua■LuaPanda ■■ PlayerAgent。

在 `service/player/player_agent.lua` 设置三个断点：

1. `REQUEST.move` 内 `skynet.call(scene_service, ...)` 这一行。
2. 下一行 `if not ok then`。
3. `player.x, player.y = rx, ry`。

启动交互客户端并登录，输入：

```
move 102 100
```

第一次停住时记录：当前 `coroutine`、`player.x/y`、`scene_service`。Step Over `skynet.call` 时，请注意 Debugger 可能进入 Runtime Lua；继续运行，直到 Response 到达后才会停在下一行。

要验证的不是“断点能停”，而是：同一个 Lua stack 跨越了消息往返；等待期间 Worker Thread 没有被阻塞，PlayerAgent 仍可接收其他类型消息。

13.2 切换到 Scene 验证 owner

停止上一次 Session，选择 Skynet Lua■LuaPanda ■■ Scene，在 `CMD.move` 中设置：

- `movement.try_move` 前；
- `grid:move`；
- `return true, x, y`。

再次执行 `move`。检查 Scene 中的 `player.move_state`、权威 `player.x/y` 与 `visible`。这一步证明判定和提交发生在 Scene，而不是 PlayerAgent。

LuaPanda 一次只注入显式选择的 Service；不要同时让多个同名实例争抢 8818 端口。

14. 调试实验二：用 Debug Console 看 Actor，而不是只看日志

正常配置启动服务器和客户端，再连接 Skynet MMO Debug Console。

依次执行：

```
list
stat
task :<PlayerAgent>
task :<Scene>
call :<Scene> "stats"
```

观察点：

- **list**：动态 PlayerAgent 和 Scene 的 Address 与启动参数。
- **stat**：CPU、消息计数、Mailbox 长度；判断是否出现 Hot Actor。
- **task**：挂起 coroutine 的 traceback；定位某条 **call** 正在等谁。
- **call ... "stats"**：读取 Scene 公开状态快照。它本身是同步 RPC，只用于隔离调试，不应高频轮询生产 Actor。

若要追一条同步调用，可在 Console 中对目标使用 **trace**，结合源码中的 source/session 观察请求和 Response。断点适合个人环境的局部控制流；**stat/task/trace** 更接近共享和生产环境允许的诊断方式。

15. 调试实验三：C Runtime 断点

使用 Skynet Runtime GDB WSL/Linux，在这些函数设置断点：

```
skynet_context_push
skynet_mq_push
skynet_context_message_dispatch
dispatch_message
```

不要试图对所有消息逐条单步，那会被 Timer、Logger、Socket 消息淹没。使用条件断点限定目标 Handle，或先从 Debug Console 查到 Scene Address。

建议回答：

1. 消息进入 `message_queue` 时，当前线程是谁？
2. 领取队列并 dispatch 的 Worker 是否固定？
3. `msg->source/session/type/size` 分别是什么？
4. `PTYPE_RESPONSE` 回到 PlayerAgent 时，Lua 层如何找到原 coroutine？

完成这组实验后，“Actor 消息调度”不再是框架术语，而是能在 C/Lua 两层观察的事实。

16. 设计练习：三个看似合理但有问题的方案

16.1 方案 A：所有战斗请求经过 CombatMgr

问题：CombatMgr 变成全服高频中心 Mailbox；不同 Scene 原本可并行的状态被重新串行化。更合理的是战斗状态由 Scene/Match 实例拥有，Combat module 作为组合模块使用；Manager 只管理实例生命周期和路由发现。

16.2 方案 B：PlayerAgent 先改坐标，再通知 Scene

问题：形成双主。Scene 可能因超速/越界拒绝，或者通知失败，持久快照与 AOI 权威状态分叉。应先让 Scene 原子校验并提交，再由 PlayerAgent 更新持久快照。

16.3 方案 C：Scene 移动中 call NavService 查询碰撞

问题：原无 yield 的提交段被拆开。恢复时 player、move quota、地图版本和旧 visible 都可能变化。若确实需要独立 NavService，应采用带地图版本的查询、恢复后 revalidate，或把确定性碰撞数据作为 Scene 本地只读数据，避免每次移动 RPC。

17. 修改 Service 前的强制审查模板

任何 Service 行为修改，先写出下面答案：

17.1 状态所有权

- 该 Service 独占写入哪些状态？
- 哪些字段只是其他 owner 的快照或缓存？
- 真相冲突时以谁为准？

17.2 执行上下文与 yield

- 入口来自 `lua/client/socket/timer/fork` 哪种协议或回调？
- 列出所有直接和间接 yield 点：`call/sleep/wait/queue` 及可能 yield 的 helper。
- yield 前读了哪些 table/object/version？
- 恢复后哪些消息可能已经修改它们？如何 revalidate？

17.3 消息选择

- 当前操作真的需要返回值吗？若不需要，为什么不是 `send`？
- 同步 RPC 链最深多少层？是否可能形成环？
- 目标退出或排队时，上游如何失败和观测？

17.4 架构与生命周期

- 是否新建了无必要 Service？普通 module 是否足够？
- Manager 是否被放进高频数据面？
- 是否制造了单点 Hot Actor？天然分片键是什么？
- fd、Service、Scene Instance、timer 是否需要 Generation/Version Check？

17.5 验证

- 哪个确定性测试覆盖失败路径和竞态窗口？
- 用哪个断点/Trace/状态快照证明消息路径？
- 用哪些 Mailbox/延迟/CPU 指标证明没有引入热点？

18. 常见误判速查

误判	实际语义	后果
Service 单线程，所以 handler 不重入	coroutine yield 后其他消息可执行	stale state、重复提交
call 阻塞 Service	只挂起当前 coroutine	误判吞吐或漏掉交错
加 Worker 就能加速一个 Hot Scene	Worker 共享调度，不会并发进入同一 Actor callback	热点仍在
Manager 多一跳只是代码更整齐	多一次序列化、排队和失败边界	中心瓶颈、尾延迟
fd 可唯一标识连接	fd 会复用	旧 close 踢掉新连接
PlayerAgent 和 Scene 都存坐标就是双主	Scene 是权威，Agent 是成功提交后的持久快照	关键在写入协议
send 成功代表业务成功	只代表完成投递尝试	静默业务失败
pcall 后返回 false 就完成容错	可能掩盖 Runtime、编程和数据错误	故障不可诊断

19. 本课源码索引

按消息从底向上阅读：

```
third_party/skynet/skynet-src/skynet_mq.c
    message_queue / global_queue / skynet_mq_push / skynet_mq_pop
```

```
third_party/skynet/skynet-src/skynet_server.c
    skynet_context / skynet_context_push
    skynet_context_message_dispatch / dispatch_message
```

```
third_party/skynet/lualib-src/lua-skynet.c
    lcallback / _cb / lsend
```

```
third_party/skynet/lualib/skynet.lua
    skynet.send / skynet.call / yield_call
    raw_dispatch_message / skynet.retpack
```

```
third_party/skynet/lualib/skynet/queue.lua
    coroutine mutex ■ wait/wakeup
```

```
service/gateway/watchdog.lua
    login ■■■■■yield ■ alive ■■■gate.forward
```

```
service/player/player_agent.lua
    ■■■■■■PTYPE_CLIENT■serial■REQUEST.move
```

```
service/scene/scene.lua
    ■■■■■■■ yield ■ move commit■■■ AOI push
```

20. 自测题

1. 为什么“一个 Service 一条 Mailbox”仍不足以证明业务 handler 原子执行？

回答必须提到：Lua dispatcher 为消息使用 coroutine；`skynet.call` 会 yield；C callback 返回后，同一 Service 可处理下一条 Mailbox 消息。

2. `skynet.call` 如何把 Response 恢复到正确 coroutine？

回答必须提到：非零 session、`session_id_coroutine`、`PTYPE_RESPONSE` 和 `resume`。

3. 为什么 Scene 的 `CMD.move` 中 `skynet.send` Push 不破坏当前提交段的原子性？

回答必须区分 send 的非等待投递与 call 的 coroutine yield；同时说明目标 PlayerAgent 的实际处理发生在之后。

4. PlayerAgent 已有 Service Mailbox，为什么还对客户端请求使用 `skynet.queue`？

回答必须说明：不同请求 handler 是不同 coroutine；首个请求跨 call yield 后，后续请求可进入；queue 把跨 yield 的业务区间互斥起来。

20.1 自测题（续）

5. 为什么坐标同时出现在 PlayerAgent 和 Scene 中却仍可维持单一权威？

回答必须说明：Scene 负责校验和实时提交，Agent 只在成功 Response 后更新持久快照，拒绝路径不写快照。

6. 如何判断一个新模块是否应拆成 Service？

回答应从状态所有权、生命周期、隔离、真实并行和资源边界论证，并计算消息/排队/失败成本。

7. 找到一个 `skynet.call` 后，代码评审至少还要追什么？

回答应覆盖：yield 前读取、恢复后 revalidate、目标退出、RPC 深度/环、是否必须要返回值、可观测性。

8. 增加 Worker Thread 为什么通常不能修复单个 Scene Mailbox 堆积？

回答必须说明：调度单位是 Service Queue，同一个 Service callback 不会因 Worker 增多而并发执行；应 profile Actor 内成本或按实例/空间进行合理分片。

21. 本课完成标准与第三课入口

不要以“读完 PDF”作为完成。你应当能够：

1. 白板画出 Service Queue、Global Queue、Worker、Lua State 和 coroutine 的关系。
2. 在 LuaPanda 中观察 PlayerAgent `call(Scene)` 的挂起与恢复。
3. 在 Debug Console 中找到 PlayerAgent/Scene，解释 `stat/task` 输出。
4. 对 Watchdog 登录或 PlayerAgent 离线流程完成一次逐 `yield` 审查。
5. 对一个拟新增 Service 给出“应拆/不应拆”的成本论证。
6. 不看答案解析八道自测题。

第三课将把这套模型应用到完整 MMO 业务闭环：登录与重连、进场、AOI、战斗、掉线保存、MySQL、错误补偿和客户端可见顺序。第二课解决“Runtime 与 Actor 语义是什么”，第三课解决“如何用这些语义构建一致的 MMO 状态机”。